# LAB 2

## Development Environment Setup & Core Networking Concepts in Practice

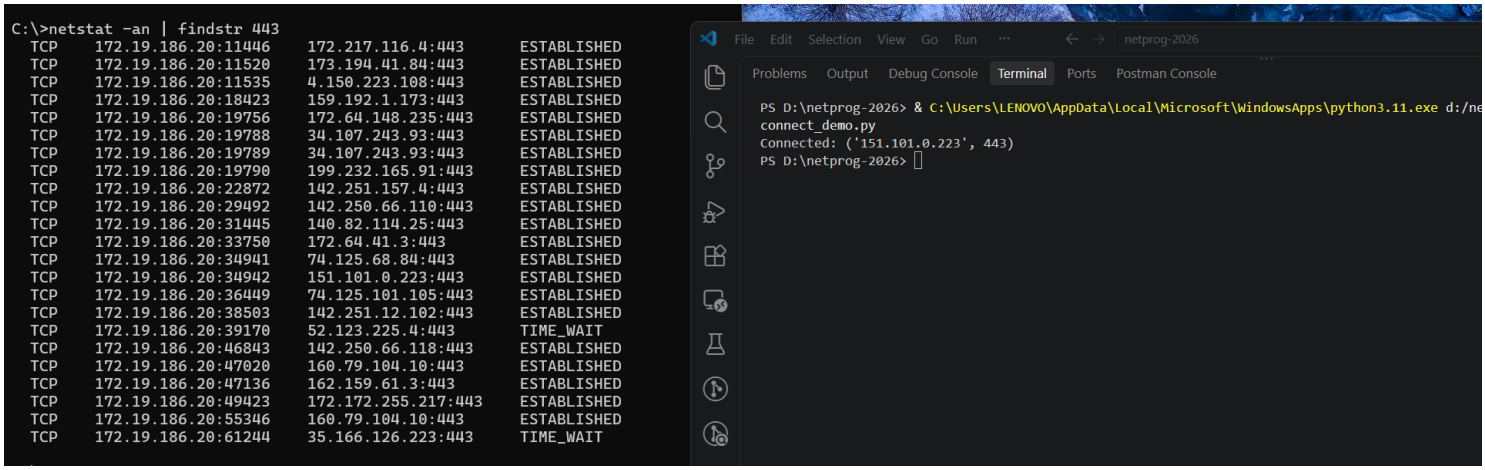
**Task 1:** Screenshot showing your (netprog-env) prompt active and python3 --version succeeding, plus your netprog-2026 repository visible on GitHub.

```
ubuntu@ubuntu:~$ python3 -m venv netprog-env
ubuntu@ubuntu:~$ source netprog-env/bin/activate
(netprog-env) ubuntu@ubuntu:~$ python3 --version
Python 3.10.12
(netprog-env) ubuntu@ubuntu:~$ git clone https://github.com/CenterHaHaYo
Cloning into 'CenterHaHaYo'...
remote: Not Found
fatal: repository 'https://github.com/CenterHaHaYo/' not found
(netprog-env) ubuntu@ubuntu:~$ git clone https://github.com/CenterHaHaYo/netprog-2026.git
Cloning into 'netprog-2026'...
Username for 'https://github.com': CenterHaHaYo
Password for 'https://CenterHaHaYo@github.com':
(netprog-env) ubuntu@ubuntu:~$ git clone https://github.com/CenterHaHaYo/netprog-2026.git
Cloning into 'netprog-2026'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 6 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
(netprog-env) ubuntu@ubuntu:~$ cd netprog-2026 && mkdir -p lab01_setup && cd lab01_setup
(netprog-env) ubuntu@ubuntu:~/netprog-2026/lab01_setup$
```

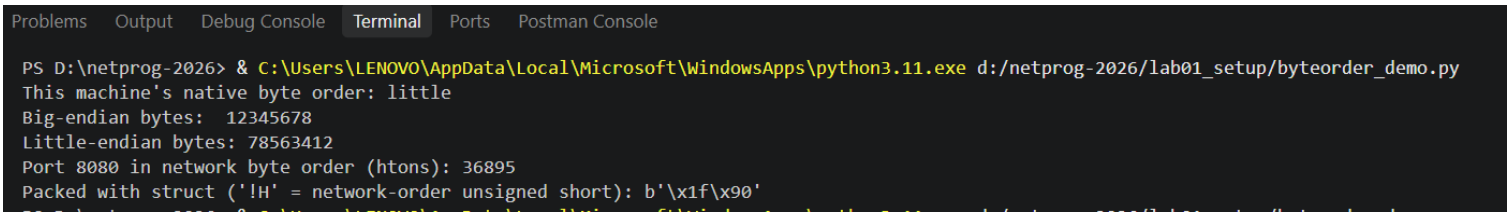
**Task 2:** Screenshot of port_lookup.py correctly classifying three different ports of your choice (one from each range: Well-Known, Registered, Dynamic/Private).

```
PS D:\netprog-2026> & C:\Users\LENOVO\AppData\Local\Microsoft\WindowsApps\python3.11.exe d:/netprog-2026/lab01_setu
p/port_lookup.py
Hostname: www.python.org
Port: 8080
www.python.org resolves to 151.101.0.223
Port 8080 is classified as: Registered
PS D:\netprog-2026> & C:\Users\LENOVO\AppData\Local\Microsoft\WindowsApps\python3.11.exe d:/netprog-2026/lab01_setu
p/port_lookup.py
Hostname: www.youtube.com
Port: 55000
www.youtube.com resolves to 142.251.152.4
Port 55000 is classified as: Dynamic/Private
PS D:\netprog-2026> & C:\Users\LENOVO\AppData\Local\Microsoft\WindowsApps\python3.11.exe d:/netprog-2026/lab01_setu
p/port_lookup.py
Hostname: www.youtube.com
Port: 80
www.youtube.com resolves to 142.251.152.4
Port 80 is classified as: Well-Known
```

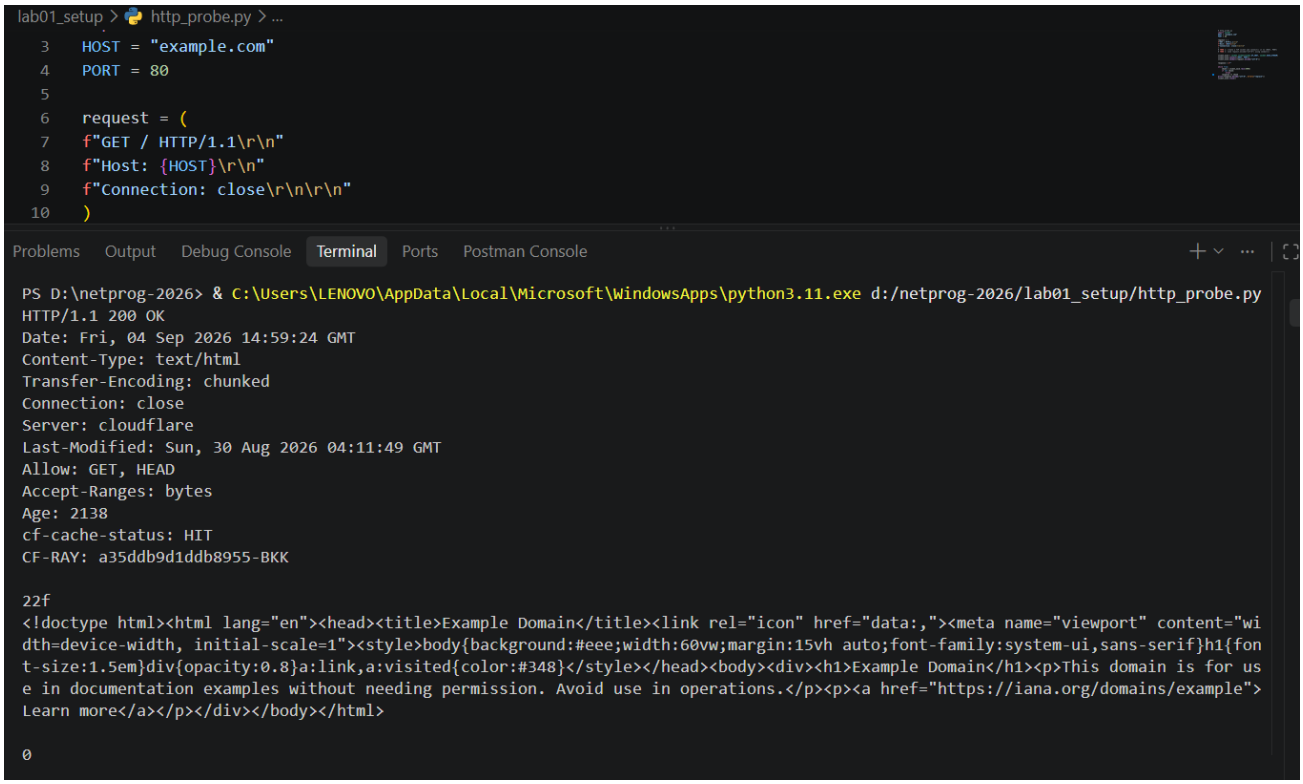
**Task 3:** Screenshot of netstat/ss (or a Wireshark capture) showing an ESTABLISHED TCP connection while connect_demo.py is sleeping.




**Task 4:** Screenshot of byteorder_demo.py's output. In 1-2 sentences, explain why the big-endian and little-endian hex strings differ but represent the same number.

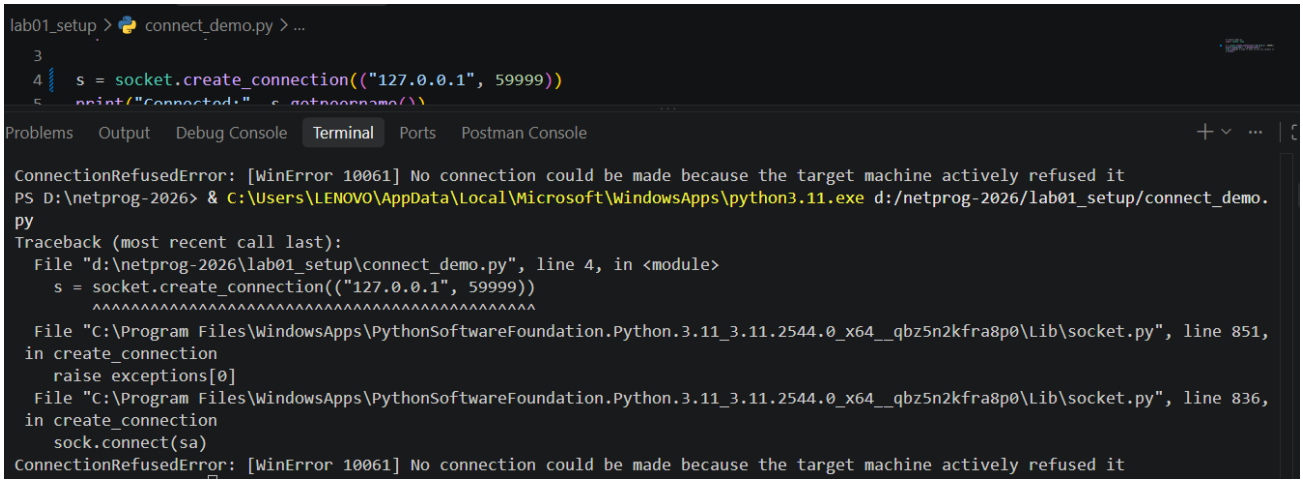


**Task 5:** Screenshot of http_probe.py successfully printing an HTTP response status line and headers from example.com.

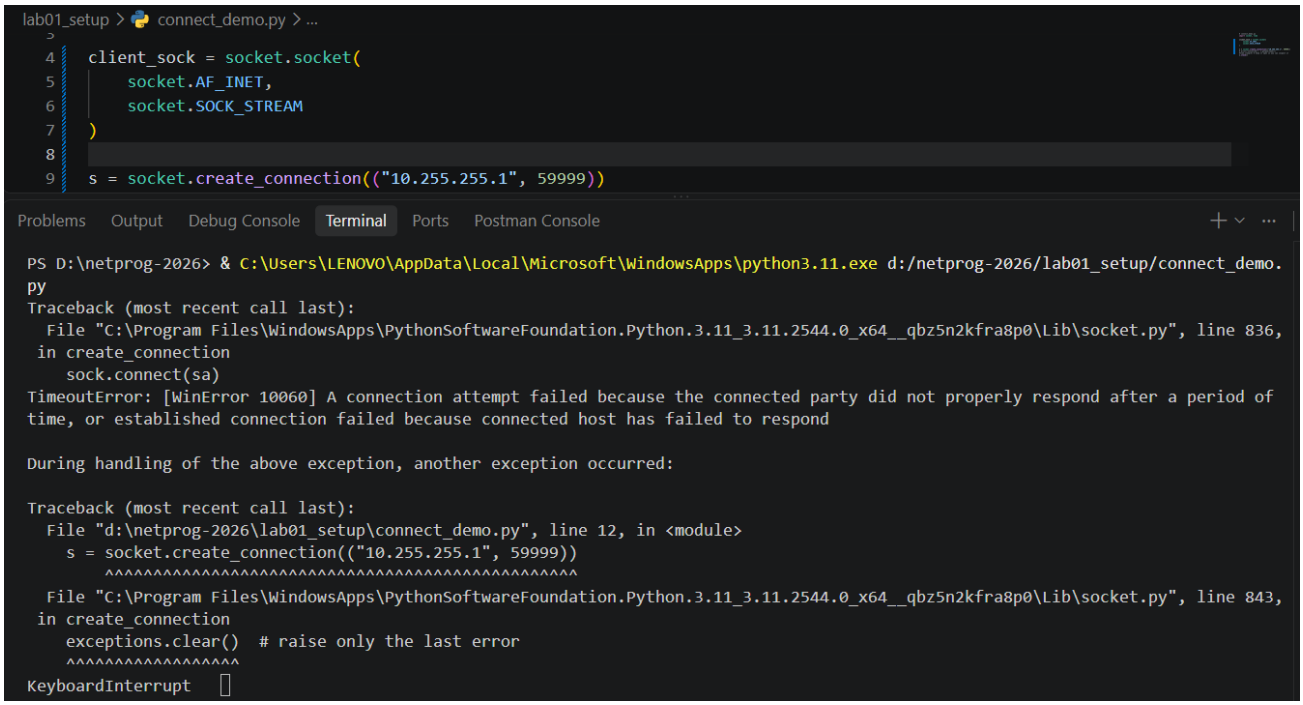



**Task 6:** Screenshots showing the fast ConnectionRefusedError, the long hang with no timeout (Ctrl+C shown), and the socket.timeout after adding settimeout(3).

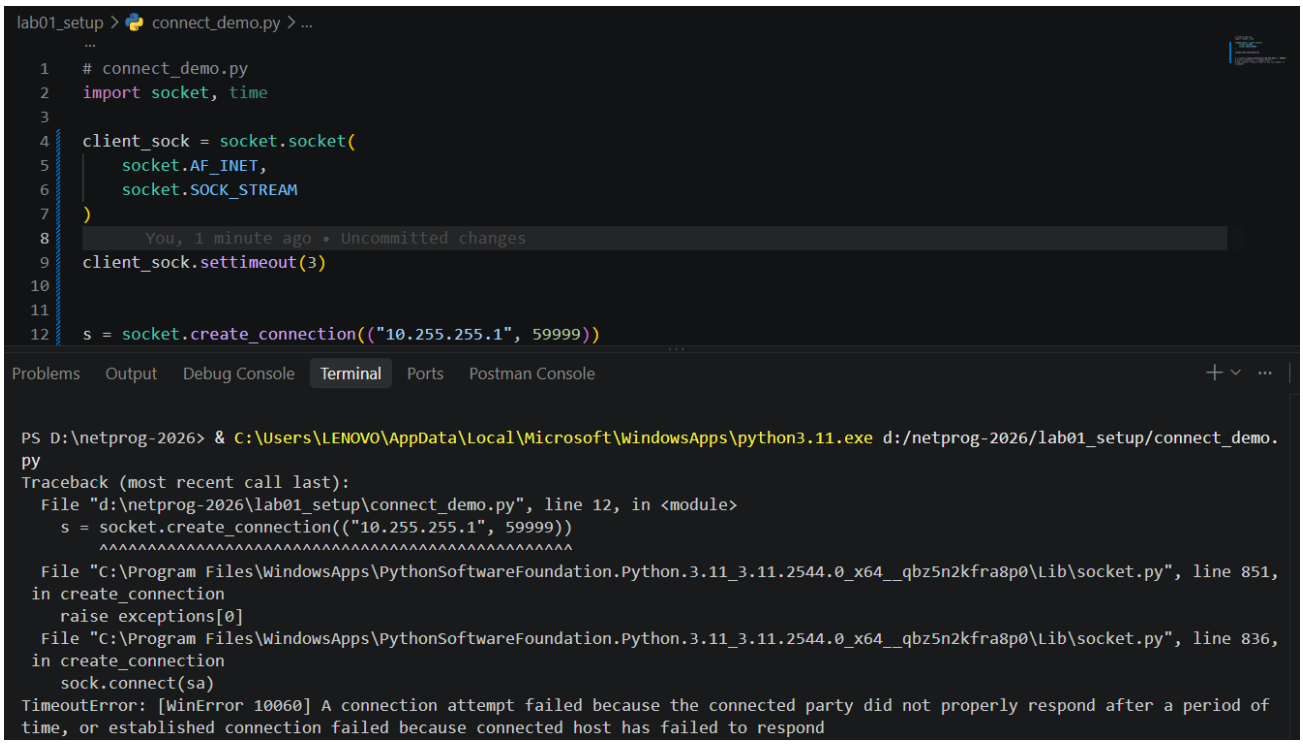
Connection Refused




Block / Hang




Adding settimeout(3) raises socket.timeout instead



## 7. Verification Checklist

| # | Checkpoint | ✓ |
|---|---|---|
| 1 | netprog-env virtual environment created and activates without errors | ☐ |
| 2 | netprog-2026 repository created on GitHub and cloned locally | ☐ |
| 3 | port_lookup.py correctly resolves a hostname and classifies all three port ranges | ☐ |
| 4 | An ESTABLISHED TCP connection to port 443 was observed while connect_demo.py was running | ☐ |
| 5 | byteorder_demo.py prints different hex byte sequences for big-endian vs little-endian | ☐ |
| 6 | http_probe.py prints a complete HTTP response (status line + headers) from a real server | ☐ |
| 7 | Removing the timeout causes a long hang; adding settimeout(3) raises socket.timeout instead | ☐ |

## 8. Troubleshooting Tips

- python3: command not found (Windows) — use py or python instead, or reinstall Python with “Add to PATH” checked.
- socket.gaierror: [Errno -2] Name or service not known — check your internet connection and that you typed the hostname correctly (no https:// prefix).
- netstat / ss shows nothing — make sure connect_demo.py is still running (it sleeps for only 5 seconds) when you check in the second terminal.
- http_probe.py hangs instead of returning — you likely forgot Connection: close in the request, so the server keeps the connection open waiting for another request.
- ConnectionRefusedError immediately in Part F step 2 instead of a hang — some networks/routers reject unreachable private addresses instantly rather than dropping packets silently; note this in your report and try a different unreachable address if you want to see the full hang.

## 9. Reflection Questions

Answer briefly (2-4 sentences each) in your report:

1. **Explain blocking I/O in the context of connect () and recv(). What is your program (and the underlying thread) doing, from the OS's point of view, while it waits in Part F?**

   Answer When connect() or recv() can't complete immediately, the OS puts the calling thread into a blocked state and removes it from the CPU scheduler's runnable queue, so it uses no CPU while waiting. The kernel sets a wake-up condition —

for connect(), arrival of a SYN-ACK or error; for recv(), arrival of data or connection close — and only resumes the thread once that condition is met. This is why in Part F, connecting to 10.255.255.1 blocks indefinitely (nothing ever answers), 127.0.0.1:59999 returns almost instantly (an immediate refusal signal), and settimeout(3) forces the kernel to wake the thread after 3 seconds regardless, raising socket.timeout.

2. **Why does socket.htons() exist at all — why can't every machine just send integers in its own native byte order?**

   Answer The socket.htons() exists because **the network needs one agreed-upon byte order**, but different CPU architectures store multi-byte numbers in memory differently. Without a conversion function like htons(), two machines with different native byte orders would misinterpret each other's numbers.

3. **In Part E, you built the "request" half of a request-response exchange by hand. Identify one thing a real HTTP client library (like Python's requests) handles for you that you had to do manually or ignored here.**

   Answer One thing request handled automatically, that http_probe.py ignores, is TLS/HTTPS negotiation. I tried to redirect the script to www.youtube.com instead of example.com and the raw socket method failed: YouTube's server expects an HTTPS connection on port 443, and even if I connect to port 80 it just sends back a 301 Moved Permanently redirect to https://www.youtube.com/ rather than any actual content. My raw socket code doesn't have the capability of doing a TLS handshake and has no logic for following redirects, so it can't actually get the page. A library like requests does the whole TLS handshake, certificate verification, and auto redirect following transparently – which is exactly the reason http_probe.py only reliably works against a plain-HTTP test server like example.com